DastarPlan — Technical Thesis Summary

Oct 17, 2026 · @Yelzhana

Recognising Cyrillic-labelled grocery products with a general-purpose vision API: the product-recognition pipeline of DastarPlan

Yelzhana Gizatova · Kazakh-British Technical University, Almaty · BSc Information and Communication Technologies (IT Management), 2026

This summary describes one component of a four-person bachelor's thesis project, *AI-Powered Nutrition Assistant: An Application for Personalized Meal Planning Using Available Ingredients* (supervisor: A. Myrzakanurov, Senior Lecturer). It is not peer-reviewed.

Team member	Role in the project
Yelzhana Gizatova (author)	**Backend developer: computer-vision and product-recognition pipeline**
Diyas Tolegenov	Backend developer: AI recipe and meal-plan generation
Dilnaz Koishybekova	UX/UI designer, project manager
Dilnaz Zhussipbekova	iOS developer

Abstract

DastarPlan is an iOS application that builds personalised meal plans from the food a user already has at home. Its first step is recognising products from a single photo. In Kazakhstan this is harder than it looks: packaging mixes Cyrillic and Latin script, general-purpose OCR reads Cyrillic less reliably, and open nutrition databases cover few local brands.

This summary describes the recognition pipeline I designed and implemented. It sends one Google Cloud Vision request with four detection features, then combines the results in three stages: object-anchored OCR for packaged goods, label mapping for fresh produce, and a keyword fallback. Recognised names are matched against Open Food Facts for verified nutrition data; names that remain unverified are normalised by GPT-3.5 Turbo into a clean Russian product name and category. The summary also sets out a planned evaluation and the pipeline's known limitations.

1. Problem

Nutrition apps lose users because entering food by hand is tedious. In the project's survey of 398 respondents, 43% wanted to track calories or macronutrients but found it too complicated, and 20.6% had tracked before but stopped. Recognising products from a photo removes most of that manual entry.

Three local conditions make recognition harder in Kazakhstan:

- **Mixed scripts.** Labels combine Cyrillic and Latin text, often in stylised fonts. General-purpose OCR returns lower-confidence and noisier results on Cyrillic, for example `MAXEER Pycckar` for a jar of mustard.
- **Sparse database coverage.** Open Food Facts, the open product database used for nutrition data, has few Kazakhstani and Central Asian brands, so many correctly read names still find no match.
- **No local training data.** There is no labelled dataset of Kazakhstani packaging, which rules out training a custom recogniser within a thesis timeline.

The design question was therefore how far a general-purpose vision API can be pushed on this packaging by adding rules, a database lookup and a language model around it, without training a model.

2. System overview

DastarPlan is a client-server system. A native iOS app (Swift, SwiftUI) captures photos and shows inventory, recipes and meal plans. A Django REST Framework backend with PostgreSQL holds users, products, inventory, recipes and meal plans, and calls three external services: Google Cloud Vision for recognition, Open Food Facts for nutrition data, and the OpenAI API for name normalisation and recipe generation.

Recognition sits at the start of the main user flow:

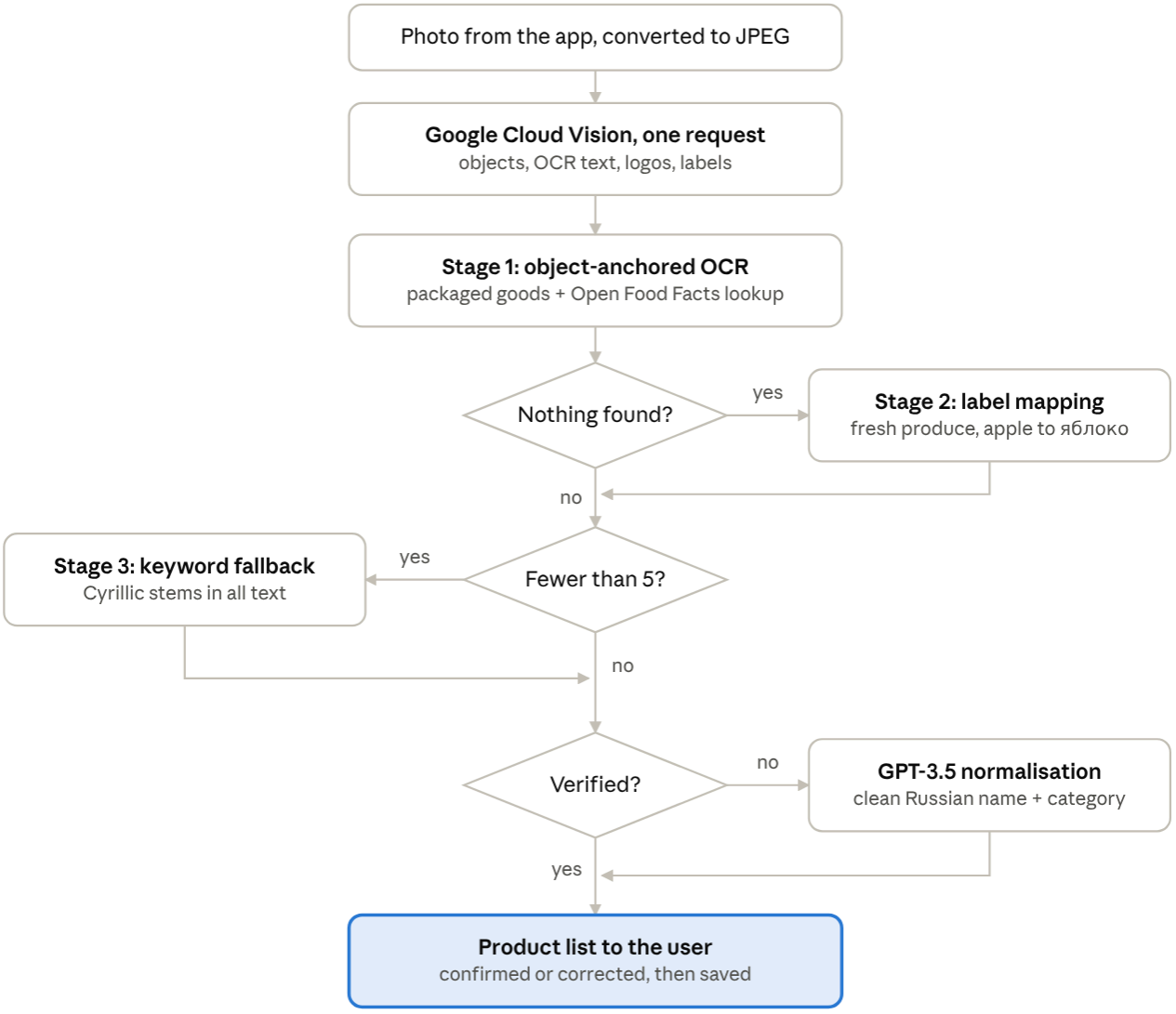
1. The user photographs products in the app (HEIC, JPG or PNG).
2. The app uploads the photo to `POST /api/scans/`.
3. The backend runs the recognition pipeline (section 3) and returns the detected products with nutrition data.
4. The user confirms or corrects the list; confirmed items are saved to their inventory.
5. Recipes and meal plans are generated from that inventory (a teammate's component, outside the scope of this summary).

The confirmation step matters: recognition is never fully accurate, so the user stays in control of what enters the inventory.

3. The recognition pipeline

The pipeline makes one Vision API call per photo, then runs up to three detection stages and two enrichment steps, each triggered only when the previous one leaves a gap. It is implemented in `myapp/cv_service.py` (entry point `analyze_image_bytes`).

Each fallback stage runs only when the previous one leaves a gap



recognition pipeline · 3 detection stages, 2 enrichment steps

Stage 2 runs only when stage 1 finds nothing, stage 3 adds products when fewer than five were found, and GPT normalises only names that Open Food Facts could not verify.

3.1 One Vision request, four features

Every photo is converted to JPEG and sent to Google Cloud Vision once, with four features enabled together to save latency and cost:

Feature	What it returns	Used for
Object localisation	Objects with bounding boxes (up to 50)	Separating products in a multi-item photo
Document text detection	OCR text organised into blocks and paragraphs	Reading product names from labels
Logo detection	Brand marks (up to 20)	Brand names when the label font is stylised
Label detection	Scene-level concepts (up to 50)	Fresh, unpackaged produce

3.2 Stage 1: object-anchored OCR for packaged goods

The main stage treats each detected object as a candidate product and attaches the text and logos that belong to it.

- Objects smaller than 0.25% of the image, or with generic names, are discarded.
- Each OCR paragraph, logo and label is assigned to the object it overlaps most (intersection-over-union, with a fallback for items fully inside a box). This keeps text from one jar off its neighbour.
- OCR lines that are mostly weights, volumes or packaging boilerplate are filtered out; the remaining lines are ranked by text height, so the largest text on the pack, usually the product name, comes first.
- The product name is built from the strongest logo plus the top OCR lines, and a product type is inferred from the object, label and OCR signals.
- A confidence score combines the three signals: 0.58 × name confidence + 0.27 × type confidence + 0.15 × object score. Candidates under 0.35 are dropped.

3.3 Stage 2: label mapping for fresh produce

If stage 1 finds nothing, the pipeline assumes unpackaged produce. Vision labels and objects above a confidence threshold (0.55 for labels, 0.35 for objects) are looked up in a dictionary that maps English labels to Russian product names and categories, for example `apple → яблоко`.

3.4 Stage 3: keyword fallback

If fewer than five products have been found, all available text (full OCR text, logo names and object names) is joined into one string and searched for trigger keywords. Each keyword list maps to one canonical product, and triggers include Cyrillic stems such as

`молок` (milk) or `творожн` (curd), so a product can be recognised even when OCR returns only a fragment of its name. The current dictionary covers 8 products and is meant to grow with real scan data.

3.5 Open Food Facts enrichment

Each name from stage 1 is first fuzzy-matched (RapidFuzz, score ⩾ 80) against a short list of common staples, then searched in Open Food Facts. A confident match adds brand, calories per 100 g, Nutri-Score and category, and marks the product as verified. Lookups are cached in memory to avoid repeat requests.

3.6 GPT-3.5 Turbo normalisation

Every product that is still unverified is sent to GPT-3.5 Turbo (temperature 0) with the raw label text and the type Vision inferred. The model returns JSON with a simple Russian product name and one of nine fixed categories, turning noisy strings like `MAXEER Pycckar` into `горчица` (mustard), category Sauces. If the call fails, the raw name is kept, so a model outage never blocks a scan.

3.7 Storage

Confirmed results are saved through three models: a `Scan` for the photo, a `Product` reused or created without duplicates, and an `InventoryItem` linking the product to the user with quantity and unit.

4. Planned evaluation

Recognition accuracy has not yet been measured. The thesis prototype was validated functionally (API tests in Postman and end-to-end flows on iOS); this section sets out the evaluation that will quantify it.

Test set. 50–100 photos of products bought in ordinary Almaty supermarkets, each photo with 1–5 items, grouped as Cyrillic-only labels, mixed or Latin labels, and fresh produce. Each item gets a ground-truth name and category recorded before testing.

Configurations. The same photos run through three versions of the pipeline, switching stages on one at a time:

1. Vision stages 1–2 only.
2. Plus the keyword fallback (stage 3).
3. Plus GPT-3.5 normalisation (full pipeline).

Metric. An item counts as recognised when the returned name refers to the same product as the ground truth (brand optional) and the category is correct. Precision is reported too, since extra wrong items also cost the user corrections.

Observed during development. The original design had two detection stages. Team tests on real kitchen photos showed they often returned fewer products than a photo contained, which is why the keyword fallback (stage 3) was added. These tests were informal, so they motivate the evaluation rather than replace it.

Expected output. Recognition rate per configuration and label type, plus precision, published with the test-set description so the results can be checked and repeated. The main question is how large the gap between Cyrillic and Latin labels is with Vision alone, and how much of it the fallback and GPT steps close.

5. Limitations and future work

The pipeline narrows the Cyrillic gap with rules and a language model but does not remove it.

- **Stylised Cyrillic fonts** remain the hardest case; when OCR returns almost nothing, neither the keyword fallback nor GPT has enough text to work with.
- **The keyword dictionary is small** (8 products) and hand-written, so stage 3 only helps for products it already knows.
- **Open Food Facts coverage is thin** for local brands. Unverified products get a clean name from GPT but no verified nutrition values.
- **Multi-item photos** depend on object localisation; small or partly hidden items can be missed, and blurry or angled photos lower OCR quality.
- **Accuracy is not yet measured** (section 4).

Next steps, in order of expected value:

1. Run the evaluation in section 4 and publish the results with the test-set description.
2. Grow the keyword dictionary and a local product table from user corrections collected at the confirmation step.
3. Collect and annotate a dataset of Kazakhstani packaging photos, then train or fine-tune a recogniser for Central Asian products.

Links

- Backend source code: github.com/yelzhana/dastarplan_app (recognition pipeline in `myapp/cv_service.py`)
- iOS client: github.com/yelzhana/dastarplan

- Demo video: https://youtube.com/shorts/_iC_jnuVxLA?feature=share
  https://youtube.com/shorts/ypjwW0PsHJk?feature=share
  https://youtu.be/EVKJm5PvHpE

Acknowledgements

I thank my teammates Diyas Tolegenov, Dilnaz Koishybekova and Dilnaz Zhussipbekova, and our supervisor A. Myrzakanurov.

References

1. Cenikj, G., Petelin, G., Koroušić Seljak, B., & Eftimov, T. (2024). Zero-shot evaluation of ChatGPT for food named-entity recognition and linking. *Frontiers in Nutrition*, 11, 1429259. doi.org/10.3389/fnut.2024.1429259
2. Dodge, S., & Karam, L. (2016). Understanding how image quality affects deep neural networks. arXiv:1604.04004. arxiv.org/abs/1604.04004
3. Google Cloud. Cloud Vision API documentation. cloud.google.com/vision/docs
4. Guan, V., et al. (2023). A novel mobile app for personalized dietary advice leveraging persuasive technology, computer vision, and cloud computing: Development and usability study. *JMIR Formative Research*. doi.org/10.2196/46839
5. Open Food Facts. Open Food Facts database and API. world.openfoodfacts.org
6. OpenAI. GPT-3.5 Turbo. platform.openai.com/docs/models